

16/9/25

Trapping Rain Water:

$H = 6, 2, 0, 4, 0, 1, 0, 5, 0, 3$

$$W[S] = \min \{ \max H[0, \dots, S-1], \max H[S+1, \dots, n] \} - H[S]$$



$\max S = \max \text{ suffixes} \rightarrow \text{from R to L}$

$\max P = \max \text{ prefixes} \rightarrow \text{from L to R}$

Sliding Window Maxima:

Given $A[1, n]$ and integer k , find the max element of each window of size k

$A = 1 \ 2 \ 3 \ 1 \ 4 \ 5 \ 2 \ 3 \ 1$

Trivial Solution:

$res = []$ $\Theta(n \cdot k)$

for $i \in [1, n-k]$:

$M = A[i]$

for $j \in [i, i+k]$

$M = \max\{M, A[j]\}$

$r.append(M)$

Better Solution:

- Insert new el
 - Delete old el
 - Compute Max
- } Max-heap
- $\leq n$

Lazy Delete: cost insert + cost extract $\Rightarrow \Theta(n \log n)$

We store pairs $\langle \text{value}, \text{position} \rangle$ in the heap

At every iteration:

- we insert new element
- while cur_max is outside the window
 - Extract max
- report cur_max

Max-Heap: ...

MAX-heap. $n = |S|$

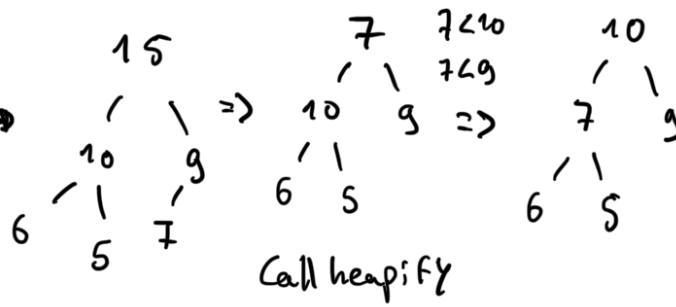
Store a (multi) Set S

- Extract Max $\longrightarrow$

- report Max $O(1)$

- Insert(x) $\Theta(\log n)$

- No delete arbitrary elements easily



Cost heapify

$\Theta(\log n)$

Predecessor problem:

BST $\rightarrow$ Binary Search Tree $\longrightarrow 1, 3, 5, 6, 8, 9, 10, 12$

Store a set S , supporting:

- Insert(x)
- Delete(x)
- Search(x)

Dictionary Problem $\Rightarrow$ Hash map



Insert(7)

$\bullet + \bullet =$ BST property

- Max/Min

- Successor(x) $\min\{y \mid y \in S \wedge y \geq x\}$

- Predecessor(x)

All ops $\Theta(\log |S|)$

SWM in $\Theta(n \log n)$ time